PHARMACY MANAGEMENT SYSTEM

Database Project Report

Prepared by: Sohila Mohamed

GitHub: sohilamohamed316 / pharmacy-db-project

May 2026

1. Introduction

This report presents the design and implementation of a relational database system for a multi-branch Pharmacy Management System. The project was developed as part of an academic database course and demonstrates the full lifecycle of database development, from conceptual modelling through physical implementation and query writing.

The system handles core pharmacy operations including medicine inventory management, sales tracking, purchase orders, supplier management, and multi-branch staff coordination.

Project Objectives

- Design a normalised relational schema for a pharmacy chain.
- Model entity relationships using an Entity-Relationship Diagram (ERD).
- Implement the schema in MySQL with proper constraints and foreign keys.
- Populate the database with realistic sample data.
- Write SQL queries covering retrieval, filtering, aggregation, and joins.

2. System Overview

The Pharmacy Management System manages data across **10 core entities** distributed over multiple branches. The system supports:

Module	Description
Medicine Catalogue	Stores medicine details, scientific names, pricing, and category
Inventory	Tracks stock levels per medicine including batch numbers and location
Categories	Groups medicines (Antibiotics, Analgesics, Vitamins, etc.)
Sales	Records customer transactions, payment mode, discount, and branch
Purchase	Manages procurement from suppliers per branch
Customers	Stores customer contact and demographic information
Pharmacists	Employee records linked to branches with salary details
Suppliers	Vendor contact information for medicine procurement
Branches	Multi-location branch details with manager assignment

3. Entity-Relationship Diagram (ERD)

The ERD below illustrates all entities, their attributes, and the relationships between them. A Person super-entity generalises Customers and Pharmacists via an ISA hierarchy. Medicines are linked to Categories and appear in both Sales and Purchase transactions through junction entities.

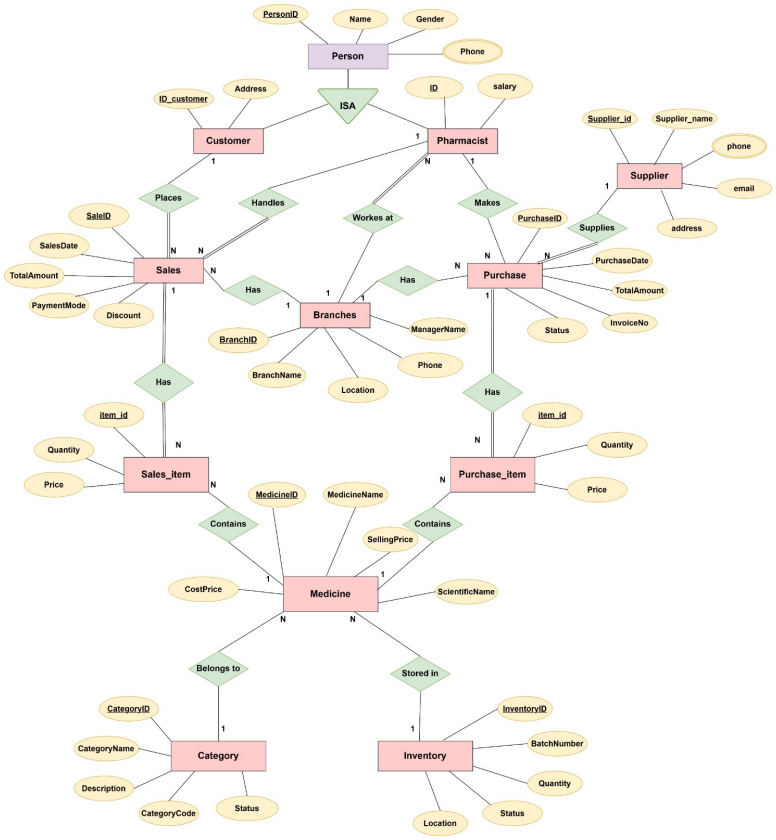

Figure 1 – Entity-Relationship Diagram

Key Relationships

Relationship	Cardinality
Person → Customer / Pharmacist	ISA (specialisation) – one-to-one
Pharmacist → Branch	Works at – many-to-one
Sales → Branch / Pharmacist / Customer	Many-to-one foreign keys
Purchase → Branch / Pharmacist / Supplier	Many-to-one foreign keys
Medicine → Category	Belongs to – many-to-one
Medicine → Inventory	Stored in – one-to-one per batch
Sales → Sales_item → Medicine	M:N resolved via junction table
Purchase → Purchase_item → Medicine	M:N resolved via junction table

4. Database Schema

The schema was implemented in MySQL. Each table uses an integer primary key; referential integrity is enforced via foreign keys. The following subsections describe each table.

Table: Categories

Column	Type	Constraint
CategoryID	INT	PRIMARY KEY

Column	Type	Constraint
CategoryName	VARCHAR(50)	NOT NULL, UNIQUE
Description	TEXT	—
CategoryCode	VARCHAR(10)	NOT NULL, UNIQUE
Status	VARCHAR(10)	DEFAULT 'ACTIVE'

Table: Medicines

Column	Type	Constraint
MedicineID	INT	PRIMARY KEY
MedicineName	VARCHAR(100)	NOT NULL
ScientificName	VARCHAR(100)	—
SellingPrice	FLOAT	NOT NULL
CostPrice	FLOAT	NOT NULL
CategoryID	INT	FK → Categories

Table: Inventory

Column	Type	Constraint
InventoryID	INT	PRIMARY KEY
BatchNumber	INT	NOT NULL
Quantity	INT	NOT NULL
Location	VARCHAR(150)	—
Status	VARCHAR(100)	—
MedicineID	INT	FK → Medicines

Table: Branches

Column	Type	Constraint
BranchID	INT	PRIMARY KEY, AUTO_INCREMENT
BranchName	VARCHAR(30)	—
Location	VARCHAR(100)	—
Phone	VARCHAR(15)	—
ManagerName	VARCHAR(50)	—

Table: Pharmacists

Column	Type	Constraint
PharmacistID	INT	PRIMARY KEY

Column	Type	Constraint
Name	VARCHAR(75)	NOT NULL
Phone	VARCHAR(15)	—
Salary	INT	NOT NULL
Gender	CHAR(1)	—
BranchID	INT	FK → Branches

Table: Customers

Column	Type	Constraint
CustomerID	INT	PRIMARY KEY
Name	VARCHAR(75)	NOT NULL
Phone	VARCHAR(15)	—
Gender	CHAR(1)	NOT NULL
Address	VARCHAR(75)	—

Table: Suppliers

Column	Type	Constraint
SupplierID	INT	PRIMARY KEY
SupplierName	VARCHAR(255)	—
Phone	VARCHAR(20)	NOT NULL
Email	VARCHAR(255)	NOT NULL
Address	VARCHAR(255)	—

Table: Sales

Column	Type	Constraint
SaleID	INT	PRIMARY KEY
SaleDate	DATE	NOT NULL
TotalAmount	INT	NOT NULL
PaymentMode	VARCHAR(20)	—
Discount	FLOAT	—
PharmacistID	INT	FK → Pharmacists
CustomerID	INT	FK → Customers
BranchID	INT	FK → Branches

Table: Purchase

Column	Type	Constraint
PurchaseID	INT	PRIMARY KEY, AUTO_INCREMENT
PurchaseDate	DATE	NOT NULL
TotalAmount	DECIMAL(10,2)	NOT NULL
InvoiceNo	VARCHAR(50)	—
Status	VARCHAR(20)	—
SupplierID	INT	FK → Suppliers
PharmacistID	INT	FK → Pharmacists
BranchID	INT	FK → Branches

5. Sample Data

The database was seeded with representative data to support testing and demonstration of all query types.

Table	Records Inserted	Notes
Branches	7	Giza, Tanta, Alexandria, Dumyat, Cairo, Mansoura
Pharmacists	7	Mixed gender, salaries 1,700 – 27,500 EGP
Customers	7	Across multiple Egyptian cities
Suppliers	9	Cities: Cairo, Mansoura, Aswan, Tanta, Giza, Alexandria
Categories	7	6 ACTIVE, 1 INACTIVE (Dermatology)
Medicines	15	Panadol, Brufen, Augmentin, Glucophage, Neurobion …
Inventory	11	Mix of Available, Low Stock, Out of Stock

Medicine Categories Breakdown

Category	Code	Status	Example Medicines
Antibiotics	ANT-01	ACTIVE	Augmentin, Zithromax, Flagyl
Analgesics	ANA-02	ACTIVE	Panadol, Brufen, Voltaren, Cataflam
Vitamins	VIT-03	ACTIVE	Neurobion
Dermatology	DER-04	INACTIVE	Clarinase, Telfast, Otrivin
Cardiology	CAR-05	ACTIVE	Aspirin Protect
Diabetes	DIA-06	ACTIVE	Amaryl, Glucophage
First Aid	FST-07	ACTIVE	—

6. SQL Queries

The following queries were implemented to demonstrate various SQL capabilities across retrieval, filtering, aggregation, update, delete, and join operations.

6.1 Basic Retrieval

Full table retrieval and projection of specific columns:

```
SELECT * FROM Pharmacists;
SELECT * FROM Customers;
SELECT Salary FROM Pharmacists;
SELECT Name   FROM Pharmacists;
```

6.2 Filtering (WHERE / LIKE / BETWEEN / IN / NOT)

WHERE – Gender filter

```
SELECT * FROM Pharmacists WHERE Gender = 'M';
SELECT * FROM CATEGORIES  WHERE STATUS = 'ACTIVE';
```

LIKE – Pattern match

```
SELECT * FROM CATEGORIES WHERE CATEGORYNAME LIKE 'A%';
```

BETWEEN – Range filter

```
SELECT * FROM CATEGORIES WHERE CATEGORYID BETWEEN 1 AND 3;
```

IN – Value list

```
SELECT * FROM Inventory WHERE Quantity IN (50, 60, 75) ORDER BY Quantity;
```

NOT – Exclusion

```
SELECT * FROM Inventory
WHERE NOT (Status = 'Low Stock' OR Status = 'Out of Stock');
```

6.3 Data Modification (UPDATE / DELETE)

```
UPDATE Pharmacists SET Phone = '01025234412' WHERE PharmacistID = 4;
UPDATE Medicines   SET SellingPrice = 15 WHERE MedicineID = 8;

DELETE FROM Pharmacists WHERE PharmacistID = 1;
DELETE FROM Sales       WHERE SaleID = 3;
```

6.4 Sorting (ORDER BY)

```
SELECT * FROM Pharmacists ORDER BY Salary;
SELECT * FROM Medicines   ORDER BY CostPrice;
```

6.5 Aggregate Functions

Function	Query	Purpose
SUM	SELECT SUM(Salary) FROM Pharmacists;	Total payroll
COUNT	SELECT COUNT(*) AS TotalBranches FROM Branches;	Number of branches
AVG (subquery)	SELECT * FROM Inventory WHERE Quantity > (SELECT AVG(Quantity) FROM Inventory);	Above-average stock

6.6 GROUP BY

```
SELECT SaleDate, SUM(TotalAmount) AS TotalAmount
FROM Sales GROUP BY SaleDate;

SELECT PaymentMode,
       SUM(TotalAmount)   AS TotalAmount,
       COUNT(PaymentMode) AS NumberOfUse
FROM Sales GROUP BY PaymentMode;
```

6.7 JOINs

INNER JOIN to retrieve medicine category names, and LEFT JOIN to include customers without any recorded sales:

```
SELECT Medicines.MedicineName, Categories.CategoryName
FROM   Medicines
INNER JOIN Categories ON Medicines.CategoryID = Categories.CategoryID;

SELECT Customers.Name, Sales.SaleID, Sales.TotalAmount
FROM   Customers
LEFT JOIN Sales ON Customers.CustomerID = Sales.CustomerID;
```

7. Conclusion

This project successfully demonstrates the design and implementation of a fully functional relational database for a multi-branch pharmacy chain. The database covers all essential operations: inventory management, sales recording, purchase tracking, and staff administration.

Key achievements of the project include:

- A well-normalised schema with 9 tables and proper foreign-key constraints.
- An ERD reflecting a clean ISA hierarchy and M:N relationship resolution.
- Realistic seed data spanning 7 branches, 15 medicines, and 9 suppliers.
- A comprehensive query set covering all major SQL clauses and functions.
- Clean, readable SQL code hosted on GitHub for version control.

> This database system provides a solid foundation for future enhancements such as prescription management, expiry-date tracking, automated reorder alerts, and a web-based front-end interface.